République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Sfax

École Nationale d'Ingénieurs de Sfax
Département de Génie Informatique
et de Mathématiques Appliquées

Rapport de Projet de Fin d'Année

Conception et Développement d'une Plateforme E-Learning Intelligente *LearnAgent*

Recommandation par IA, suivi de progression et détection des lacunes pédagogiques

Réalisé par :

Wassim Abdelhedi **Donia Bahloul** **Amin Frikha**

Soutenu publiquement le 02/05/2026 devant le jury composé de :

Mme. Sahla Masmoudi	**Encadrante**
**********	**Co-encadrant**
**********	**Examinateur**

Année Universitaire 2025–2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet de fin d'année.

Nous adressons tout d'abord nos sincères remerciements à notre encadrante, **Mme. Masmoudi Sahla**, pour son soutien indéfectible, ses conseils avisés et sa disponibilité tout au long de ce travail. Sa rigueur académique et son expertise technique nous ont été d'une valeur inestimable.

Nous remercions également l'ensemble du corps enseignant du département informatique de l'ENIS pour la qualité de la formation dispensée, qui nous a fourni les bases solides nécessaires à l'accomplissement de ce projet.

Nos remerciements s'étendent également à nos familles pour leur patience, leur encouragement et leur soutien moral, sans lesquels ce travail n'aurait pas été possible.

Enfin, nous remercions tous nos camarades de promotion pour l'entraide et l'esprit d'équipe qui ont caractérisé notre parcours de formation.

BAHLOUL Donia, FRIKHA Amin, ABDELHEDI Wassim

Liste des Abréviations

Abréviation	Signification
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CORS	Cross-Origin Resource Sharing
DTO	Data Transfer Object
ENIS	École Nationale d'Ingénieurs de Sfax
GPT	Generative Pre-trained Transformer
HTTP	HyperText Transfer Protocol
IA	Intelligence Artificielle
JPA	Java Persistence API
JWT	JSON Web Token
KPI	Key Performance Indicator
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PFA	Projet de Fin d'Année
QCM	Questionnaire à Choix Multiples
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
UML	Unified Modeling Language

Résumé

Ce rapport présente le développement de LearnAgent, une plateforme e-learning intelligente réalisée dans le cadre du Projet de Fin d'Année à l'ENIS. Elle vise à offrir une expérience d'apprentissage personnalisée face à la diversité des ressources éducatives en ligne.

La plateforme propose des fonctionnalités principales telles qu'un système d'authentification sécurisé (JWT), un moteur de gestion de cours avec suivi de progression et des quiz interactifs pour l'évaluation. Sa valeur ajoutée repose sur l'intégration de l'intelligence artificielle, notamment un système de recommandation basé sur Sentence-BERT et un tuteur intelligent utilisant l'API Google Gemini, permettant d'adapter les contenus et de générer des explications personnalisées.

Sur le plan technique, LearnAgent s'appuie sur une architecture microservices combinant Spring Boot, React.js et FastAPI, et a été développé selon la méthodologie agile Scrum.

Mots-clés : E-learning, Intelligence Artificielle, Recommandation, Sentence-BERT, NLP, Microservices, Spring Boot, React.js.

Abstract

This report presents the development of LearnAgent, an intelligent e-learning platform carried out as part of the Final Year Project at ENIS. The platform aims to provide a personalized and structured learning experience in response to the growing diversity of online educational resources.

LearnAgent includes core features such as secure authentication (JWT), course management with progress tracking, and interactive quizzes for assessment. Its main contribution lies in the integration of artificial intelligence, including a semantic recommendation system based on Sentence-BERT and an intelligent tutor powered by the Google Gemini API, enabling personalized guidance, learning gap detection, and adaptive explanations.

The platform is built on a microservices architecture combining Spring Boot, React.js, and FastAPI, and developed following the Agile Scrum methodology.

Keywords : E-learning, Artificial Intelligence, Recommendation, Sentence-BERT, NLP, Microservices, Spring Boot, React.js.

Table des matières

Introduction Générale **1**

1 Étude Préalable et Spécification des Besoins **2**
Introduction 2
1.1 Présentation du projet 2
1.1.1 Contexte et motivation 2
1.1.2 Périmètre du projet 3
1.2 Étude de l'existant 3
1.2.1 Analyse des plateformes existantes 3
1.2.2 Critique de l'existant 3
1.3 Méthodologie adoptée : Scrum 4
1.3.1 Présentation de Scrum 4
1.3.2 Rôles Scrum dans notre équipe 4
1.3.3 Cérémonies Scrum 5
1.3.4 Product Backlog 5
1.4 Spécification des besoins 5
1.4.1 Identification des acteurs 5
1.4.2 Diagramme de cas d'utilisation global 6
1.4.3 Besoins fonctionnels 7
1.4.4 Besoins non fonctionnels 8
Conclusion 8

2 Conception Architecturale du Système **10**
Introduction 10
2.1 Architecture globale 10
2.1.1 Vue d'ensemble 10
2.1.2 Diagramme d'architecture 10
2.1.3 Diagramme de cas d'utilisation – Authentification 11
2.2 Architecture du Backend (Spring Boot) 12
2.2.1 Structure en couches N-Tier 12
2.2.2 Modèle de données (17 entités JPA) 12
2.2.3 Sécurité – Architecture JWT 13

2.3 Architecture du Frontend (React.js) 13
2.3.1 Structure de l'application 13
2.3.2 Design System 14
2.4 Architecture du Moteur IA (Python FastAPI) 14
2.4.1 Structure du service 14
Conclusion 15

3 Mise en œuvre incrémentale de la plateforme LearnAgent 16
Introduction 16
3.1 Sprint 1 – Authentification et gestion des utilisateurs 16
3.1.1 Planification du sprint 16
3.1.2 Réalisations techniques 17
3.1.3 Bilan du sprint 18
Captures d'écran – Sprint 1 19
3.2 Sprint 2 – Gestion des cours multi-chapitres 20
3.2.1 Planification du sprint 20
3.2.2 Réalisations techniques 20
3.2.3 Bilan du sprint 21
Captures d'écran – Sprint 2 22
3.3 Sprint 3 – Fonctionnalités pédagogiques avancées 23
3.3.1 Planification du sprint 23
3.3.2 Module d'Évaluation : Quiz 24
3.3.3 Module d'Activités Pratiques : Exercices 25
3.3.4 Bilan du sprint 26
Captures d'écran – Sprint 3 26
Conclusion 28

4 Sprints 4 et 5 – Intelligence Artificielle et Messagerie Avancée 29
Introduction 29
4.1 Sprint 4 – Intelligence Artificielle et Recommandation Sémantique 29
4.1.1 Architecture du Service IA et Choix technologique : Sentence-BERT 29
4.1.2 Sprint Backlog – Sprint 4 30
4.2 Pipeline de Recommandation Hybride 30
4.2.1 Flux complet de recommandation 30
4.2.2 Algorithme de Scoring Hybride 31
4.2.3 Expansion de Requête et NLP Bilingue 32
4.2.4 Intégration Backend (SearchService) 32
4.3 Détection des Lacunes (Weak Topic Detector) 32
4.3.1 Algorithme de détection 32
4.4 Tuteur Pédagogique IA (Gemini) 33

4.5 Endpoints REST du Service IA 33
4.5.1 Tableau de Bord Analytique et Visualisation de Performance 34
4.6 Sprint Review – Sprint 4 35
4.7 Sprint 5 – Système de Messagerie Intelligente 36
4.7.1 Architecture du module de messagerie 36
4.7.2 Intégration Frontend 36
Conclusion 37

Références Bibliographiques **38**

A Tableau récapitulatif des Endpoints REST Backend **39**

B Stack Technologique **41**

Conclusion Générale et Perspectives **43**

Table des figures

1.1 Diagramme Roadmap du Product Backlog priorisé par Sprint 5
1.2 Diagramme de cas d'utilisation global – Plateforme LearnAgent 6

2.1 Architecture globale du système LearnAgent 11
2.2 Diagramme de cas d'utilisation – Module Authentification 11
2.3 Architecture N-Tier du backend Spring Boot 12
2.4 Architecture modulaire du frontend React.js 14
2.5 Architecture du microservice Python FastAPI (moteur IA) 15

3.1 Architecture détaillée du mécanisme d'authentification 18
3.2 Gestion des sessions et contrôle d'accès (Frontend) 18
3.3 Capture d'écran : Page d'inscription (Register) 19
3.4 Capture d'écran : Page de connexion (Login) 19
3.5 Capture d'écran : Tableau de bord Administrateur – Gestion des rôles . . . 19
3.6 Architecture relationnelle des cours multi-chapitres et suivi de progression . 21
3.7 Algorithme de contrôle d'accès basé sur la progression de l'étudiant 21
3.8 Capture d'écran : Interface de création d'un cours (vue Enseignant) 22
3.9 Capture d'écran : Visualisation des chapitres et jauge de progression (vue Étudiant) 22
3.10 Capture d'écran : Message de blocage d'accès (progression incomplète) . . 23
3.11 Modèle architectural du module d'évaluation par Quiz 25
3.12 Pipeline transactionnel de correction automatique d'un Quiz 25
3.13 Logique de liaison et de suivi du module Exercice 26
3.14 Capture d'écran : Interface de création d'un quiz QCM (vue Enseignant) . 26
3.15 Capture d'écran : Passage d'un quiz interactif (vue Étudiant) 27
3.16 Capture d'écran : Résultat et score après soumission du quiz 27
3.17 Capture d'écran : Interface de soumission d'un exercice pratique (vue Étudiant) 27
3.18 Capture d'écran : Tableau de bord analytique Enseignant (suivi des étudiants) 28

4.1 Répartition des tâches – Sprint Backlog Sprint 4 30
4.2 Schéma explicatif du flux complet de recommandation hybride LearnAgent 31
4.3 Schéma du processus de détection automatique des lacunes pédagogiques . 33

4.4 Endpoints REST du microservice IA FastAPI 34
4.5 Architecture du moteur analytique et de visualisation des performances . . 35
4.6 Architecture du module de messagerie – Sprint 5 36

Liste des tableaux

1.1 Comparaison des plateformes e-learning existantes 3
1.2 Répartition des rôles Scrum 4
1.3 Besoins non fonctionnels du système 8

3.1 Sprint Backlog – Sprint 1 (Authentification & Sécurité) 17
3.2 Sprint Backlog – Sprint 2 (Moteur de Cours) 20
3.3 Sprint Backlog – Sprint 3 (Quiz & Évaluation) 24

A.1 Principaux endpoints REST du backend Spring Boot (port 8081) 39

B.1 Technologies utilisées dans le projet LearnAgent 41

Introduction Générale

L'ère numérique a profondément transformé le secteur de l'éducation. L'apprentissage en ligne (e-learning) est devenu une réalité incontournable, offrant aux apprenants une flexibilité et une accessibilité sans précédent. Face à la diversité croissante des contenus disponibles, il devient cependant difficile pour un étudiant de trouver les ressources les mieux adaptées à ses besoins spécifiques, à son niveau et à ses objectifs d'apprentissage. Cette situation met en évidence la nécessité de développer des plateformes intelligentes, capables non seulement de centraliser les ressources pédagogiques, mais aussi de guider chaque apprenant de manière personnalisée.

C'est dans ce contexte que s'inscrit notre Projet de Fin d'Année, qui consiste à concevoir et à développer **LearnAgent**, une plateforme e-learning intelligente. Notre objectif est d'offrir une expérience d'apprentissage structurée, progressive et personnalisée, grâce notamment à un moteur de recommandation intelligent basé sur l'intelligence artificielle. Pour répondre à ces ambitions, nous avons conçu une architecture logicielle robuste suivant les meilleures pratiques du génie logiciel, intégrant un système d'authentification sécurisé par tokens JWT, un moteur de cours multi-chapitres avec suivi de progression strict, des quiz interactifs avec détection automatique des lacunes par IA, ainsi qu'un système de notification intelligent pour améliorer l'engagement des apprenants. L'ensemble du projet a été conduit selon la méthodologie agile Scrum afin d'assurer une livraison itérative et incrémentielle de valeur.

Le présent rapport rend compte de l'ensemble des travaux réalisés et est structuré en cinq chapitres complémentaires. Le premier chapitre propose une étude préalable incluant l'analyse de l'existant, la présentation de la méthodologie Scrum adoptée et la spécification des besoins fonctionnels et non fonctionnels. Le deuxième chapitre est consacré à la conception architecturale globale du système. Le troisième chapitre couvre les deux premiers sprints portant sur l'authentification et la gestion des cours multi-chapitres. Le quatrième chapitre présente le troisième sprint relatif aux fonctionnalités pédagogiques (quiz et exercices). Enfin, le cinquième chapitre expose les sprints quatre et cinq dédiés au moteur de recommandation IA, à la détection des lacunes et au système de messagerie avancée.

Chapitre 1

Étude Préalable et Spécification des Besoins

Introduction

Ce premier chapitre pose les fondements du projet **LearnAgent**. Nous commençons par présenter le contexte et les motivations qui ont conduit à l'émergence de cette idée, puis nous analysons les solutions existantes afin d'en dégager les limites. Nous exposons ensuite la méthodologie de gestion de projet adoptée, avant de détailler la spécification complète des besoins fonctionnels et non fonctionnels du système, en identifiant les différents acteurs impliqués.

1.1 Présentation du projet

1.1.1 Contexte et motivation

La démocratisation d'Internet et la montée en puissance des technologies mobiles ont créé un terrain fertile pour le développement de solutions éducatives numériques. Les plateformes e-learning de référence telles que Coursera, Udemy ou Moodle ont déjà démontré l'immense potentiel de ce marché et l'appétit des apprenants pour des contenus accessibles en ligne. Cette dynamique mondiale constitue le terreau idéal pour proposer une solution nouvelle, pensée pour répondre aux exigences d'une pédagogie moderne, intelligente et personnalisée.

Notre projet **LearnAgent** s'inscrit dans cette tendance en proposant une plateforme e-learning innovante qui va au-delà de la simple mise à disposition de contenus. Elle offre une gestion de cours multi-chapitres avec progression stricte, des quiz interactifs couplés à un moteur de détection automatique des lacunes par intelligence artificielle, et un moteur de recommandation intelligent exploitant des techniques d'intelligence artificielle pour orienter chaque étudiant vers les ressources les plus pertinentes en fonction de son profil et de ses besoins.

1.1.2 Périmètre du projet

La plateforme LearnAgent s'adresse à trois types d'utilisateurs distincts, chacun disposant d'un espace et de fonctionnalités dédiés. Les **Administrateurs** assurent la gestion globale de la plateforme, des utilisateurs et des catégories de cours. Les **Enseignants** créent et gèrent des cours structurés en chapitres (avec supports PDF, vidéo, PPTX) ainsi que des exercices et des quiz QCM. Les **Étudiants**, quant à eux, s'inscrivent aux cours, progressent chapitre par chapitre selon un ordre strict, accèdent aux quiz et exercices uniquement après avoir complété l'ensemble des chapitres, et bénéficient de recommandations personnalisées basées sur l'IA.

1.2 Étude de l'existant

1.2.1 Analyse des plateformes existantes

Avant d'entamer le développement, nous avons procédé à une analyse comparative de plusieurs plateformes e-learning existantes afin d'identifier leurs points forts et leurs limites, comme le résume le tableau 1.1 ci-dessous.

Table 1.1 – Comparaison des plateformes e-learning existantes

Critère	**Moodle**	**Udemy**	**LearnAgent**
Open Source	Oui	Non	Oui
Recommandation IA	Non	Partielle	Oui (IA sémantique)
Détection des lacunes	Non	Non	Oui (IA)
Cours multi-chapitres	Oui	Oui	Oui
Progression stricte	Partielle	Non	Oui
API REST moderne	Limitée	Oui	Oui
Gestion des quiz QCM	Oui	Limitée	Oui + correction auto
Architecture microservices	Non	Oui	Oui

1.2.2 Critique de l'existant

L'analyse comparative menée révèle plusieurs lacunes récurrentes dans les solutions existantes : une architecture monolithique difficile à maintenir et à faire évoluer, un manque de personnalisation basée sur l'IA sémantique, une absence de détection automatique des lacunes suite à un échec à un quiz, des interfaces utilisateurs vieillissantes et peu réactives, ainsi qu'une absence de progression structurée et obligatoire chapitre par chapitre. Notre solution **LearnAgent** apporte des réponses concrètes à ces lacunes grâce à une architecture microservices moderne, un moteur d'intelligence artificielle pour la recommandation

personnalisée et la détection automatique des points faibles, ainsi qu'un mécanisme de progression stricte garantissant un parcours d'apprentissage pleinement structuré.

1.3 Méthodologie adoptée : Scrum

1.3.1 Présentation de Scrum

Pour la gestion de ce projet, nous avons adopté la méthodologie agile **Scrum**. Cette méthode itérative et incrémentielle est particulièrement adaptée aux projets de développement logiciel où les exigences peuvent évoluer au cours du temps. Elle permet de livrer des fonctionnalités de manière régulière et d'intégrer les retours au fil des itérations, appelées *sprints*.

Scrum repose sur trois piliers fondamentaux :

- **Transparence** : Toute l'équipe dispose d'une vision claire et partagée de l'avancement du projet.
- **Inspection** : Des points réguliers (cérémonies) permettent d'évaluer la progression et la qualité du travail produit.
- **Adaptation** : L'équipe s'ajuste en continu en fonction des retours obtenus lors des inspections.

1.3.2 Rôles Scrum dans notre équipe

La répartition des rôles au sein de notre équipe a été définie de façon à tirer le meilleur parti des compétences de chacun, comme le présente le tableau 1.2 suivant.

Table 1.2 – Répartition des rôles Scrum

Rôle	**Membre**	**Responsabilités principales**
Product Owner	Mme. Masmoudi Sahla	Définition de la vision produit, validation des livrables, priorisation du backlog
Scrum Master	FRIKHA Amin	Animation des cérémonies Scrum, levée des obstacles, coordination d'équipe
Dev Team	BAHLOUL Donia	Frontend React.js, conception UI/UX, interfaces utilisateurs
Dev Team	ABDELHEDI Wassim	Backend Spring Boot, moteur IA Python, intégration système
Dev Team	FRIKHA Amin	Backend Spring Boot, sécurité JWT, API REST

1.3.3 Cérémonies Scrum

Tout au long du projet, les cérémonies Scrum suivantes ont été rigoureusement appliquées. Le **Sprint Planning** est organisé au début de chaque sprint (d'une durée de deux semaines) pour planifier et répartir les tâches à réaliser. Le **Daily Scrum** est une réunion quotidienne de quinze minutes permettant de synchroniser l'équipe et d'identifier rapidement les obstacles. La **Sprint Review** est conduite en fin de sprint pour démontrer les fonctionnalités réalisées au Product Owner et recueillir ses retours. Enfin, la **Sprint Retrospective** permet à l'équipe d'analyser les points positifs et les axes d'amélioration pour le sprint suivant.

1.3.4 Product Backlog

Le Product Backlog regroupe l'ensemble des User Stories identifiées et priorisées par le Product Owner. Chaque story décrit une fonctionnalité du point de vue de l'utilisateur final, avec une priorité et un sprint de réalisation associé, comme l'illustre la figure 1.1 ci-dessous.

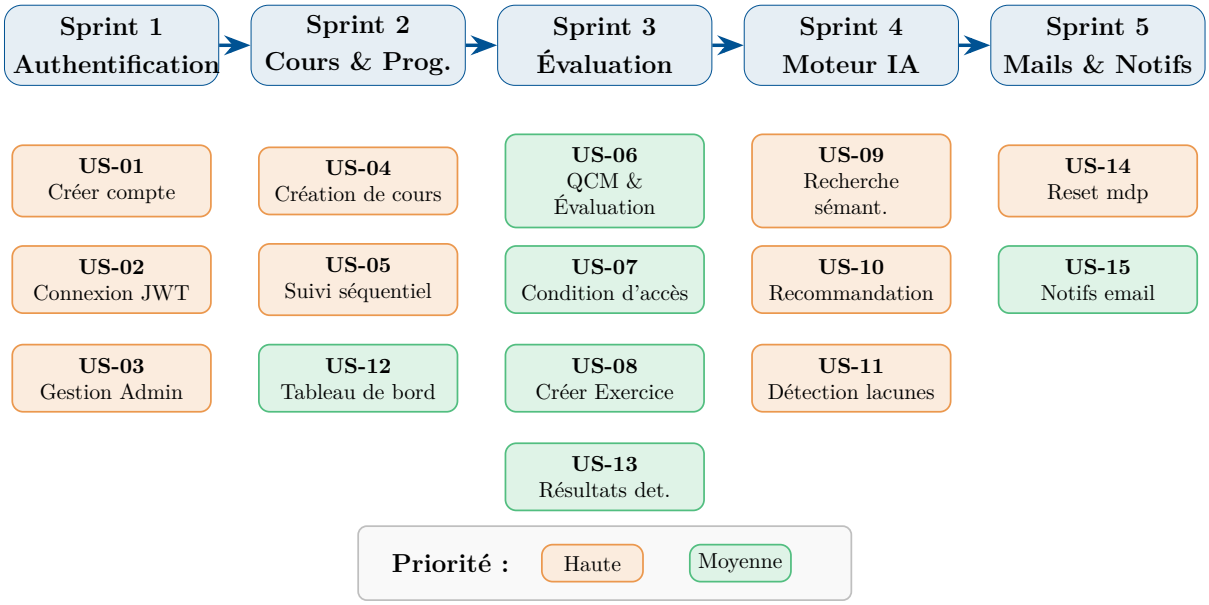

FIGURE 1.1 – Diagramme Roadmap du Product Backlog priorisé par Sprint

1.4 Spécification des besoins

1.4.1 Identification des acteurs

Un acteur représente toute entité externe au système qui interagit avec lui, qu'il s'agisse d'une personne, d'un groupe ou d'un système tiers. Dans le cadre de la plateforme **Lear-**

nAgent, trois acteurs principaux ont été identifiés.

L'Étudiant (STUDENT) est l'acteur central de la plateforme. Il s'inscrit sur la plateforme, s'abonne à des cours et progresse chapitre par chapitre selon un parcours strictement ordonné. Il accède aux quiz et exercices uniquement après avoir complété l'ensemble des chapitres d'un cours. Il bénéficie également du moteur de recommandation IA pour découvrir des cours adaptés à ses besoins, et reçoit des notifications lors des mises à jour de son parcours d'apprentissage.

L'Enseignant (TEACHER) est responsable de la création et de la gestion des contenus pédagogiques. Il crée des cours structurés en chapitres ordonnés, associés à des supports multimédias variés (PDF, vidéo, PPTX). Il conçoit également des quiz QCM et des exercices pratiques, et suit les résultats et la progression de ses étudiants.

L'Administrateur (ADMIN) dispose d'un accès complet à l'ensemble de la plateforme. Il gère les comptes utilisateurs (activation, désactivation, attribution des rôles), supervise les catégories de cours et assure la bonne configuration globale du système.

1.4.2 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation global, présenté à la figure 1.2, offre une vue synthétique des interactions entre les différents acteurs et le système LearnAgent.

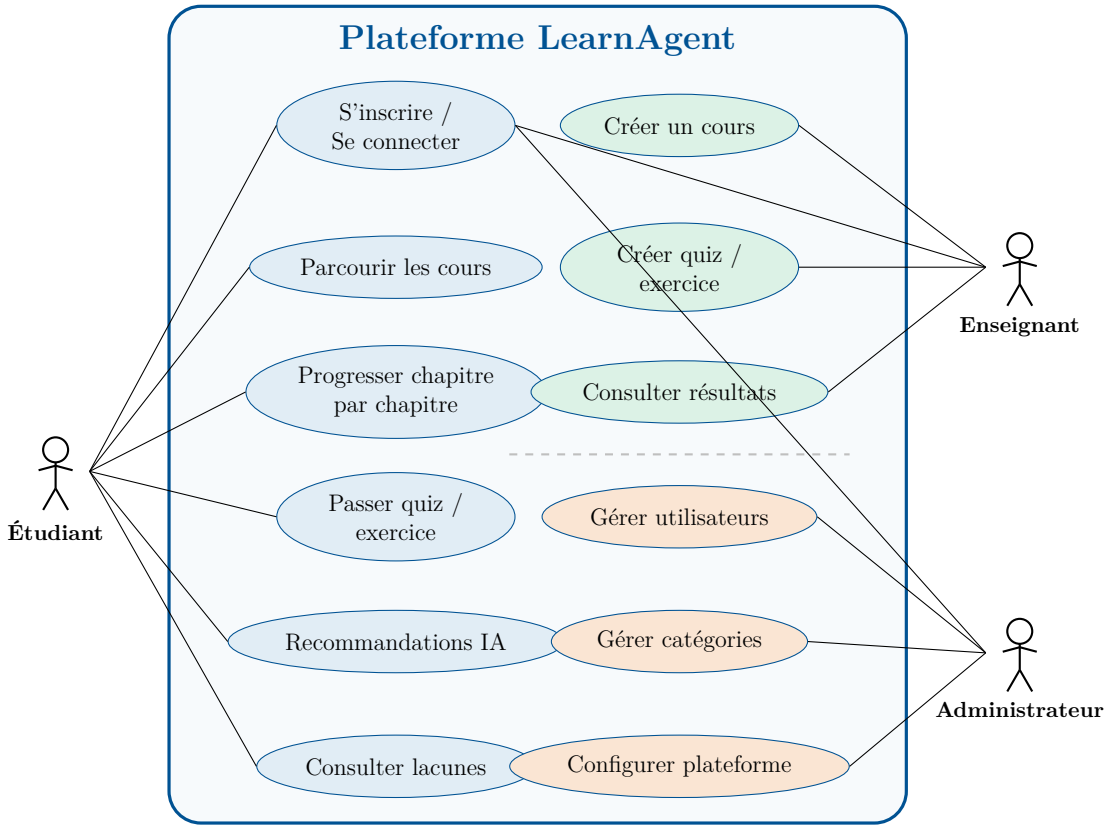

FIGURE 1.2 – Diagramme de cas d'utilisation global – Plateforme LearnAgent

1.4.3 Besoins fonctionnels

Les besoins fonctionnels ont été regroupés en quatre modules correspondant aux grandes fonctionnalités de la plateforme.

Module Authentification

— Inscription d'un nouvel utilisateur (étudiant ou enseignant).
— Connexion avec génération d'un token JWT (validité 24h).
— Gestion des rôles (ADMIN, TEACHER, STUDENT).
— Déconnexion et invalidation du token côté client.

Module Cours Multi-Chapitres

— Création, modification et suppression de cours par les enseignants.
— Structure des cours en chapitres ordonnés (`position`).
— Support multimédias variés par chapitre (PDF, vidéo MP4, PPTX, image) via l'enum `SupportType`.
— Catégorisation et classification par niveau de difficulté (BEGINNER, INTERMEDIATE, ADVANCED).
— Suivi de progression strict : un chapitre ne peut être accédé que si le précédent est complété (`ChapterProgress`).
— Inscription des étudiants aux cours et calcul automatique du pourcentage de progression.
— Accès aux quiz et exercices uniquement après complétion de 100% des chapitres.

Module Quiz et Exercices

— Création de quiz avec questions à choix multiples (QCM) par les enseignants.
— Soumission de réponses et calcul automatique du score en pourcentage.
— Historique des résultats (`QuizResult`) et statistiques de performance.
— Après un échec : appel au moteur IA pour détecter les topics non maîtrisés et les mapper aux chapitres.
— Création d'exercices pratiques avec fichiers téléchargeables.
— Suivi de complétion des exercices (`ExerciseCompletion`).

Module Recommandation IA

— Analyse sémantique du contenu des cours à l'aide d'un modèle de traitement du langage naturel (NLP) multilingue.

— Génération de recommandations personnalisées en fonction de la requête et du profil d'apprentissage de l'étudiant.
— Recherche intelligente de cours avec extraction automatique de mots-clés pertinents.
— Détection des lacunes : identification automatique des chapitres à réviser suite à un échec, par analyse des réponses incorrectes soumises.
— Extraction automatique de texte des supports de cours (PDF, DOCX, PPTX) pour l'indexation sémantique.
— Historique et traçabilité des recherches effectuées par chaque étudiant.

Module Notification et Messagerie Avancée

— Validation des adresses email à l'inscription.
— Récupération sécurisée des mots de passe par email (génération de tokens uniques et liens à durée de vie limitée).
— Envoi de notifications automatiques lors des mises à jour de cours et de leçons pour les étudiants inscrits.
— Relances par email pour les étudiants inactifs (`EmailScheduler`).

1.4.4 Besoins non fonctionnels

En complément des besoins fonctionnels, le tableau 1.3 synthétise les exigences non fonctionnelles auxquelles la plateforme doit répondre.

Table 1.3 – Besoins non fonctionnels du système

Critère	**Exigence**
Performance	Temps de réponse API inférieur à 500 ms pour les endpoints standard
Sécurité	Authentification JWT (JJWT 0.12.5), chiffrement des mots de passe (BCrypt), CORS configuré
Scalabilité	Architecture microservices permettant l'indépendance du service IA
Disponibilité	Base de données hébergée sur Supabase (PostgreSQL cloud)
Utilisabilité	Interface responsive Dark Mode (desktop et mobile)
Maintenabilité	Code documenté, architecture en couches N-Tier
Upload fichiers	Support jusqu'à 500 MB par fichier (PDF, MP4, PPTX)

Conclusion

Ce premier chapitre a permis de poser les bases du projet **LearnAgent** en définissant clairement son contexte, ses motivations et son périmètre d'application. L'analyse de l'existant a mis en évidence les lacunes des solutions actuelles et a justifié les choix technologiques et fonctionnels de notre plateforme. La méthodologie Scrum, décrite dans ce chapitre, nous a fourni un cadre de travail agile et structuré, tandis que la spécification des

besoins a permis d'identifier avec précision les fonctionnalités à développer et les acteurs impliqués. Ces éléments constituent le socle sur lequel repose la conception architecturale présentée dans le chapitre suivant.

Chapitre 2

Conception Architecturale du Système

Introduction

Ce deuxième chapitre est dédié à la conception architecturale du système **LearnAgent**. Après avoir défini les besoins dans le chapitre précédent, nous présentons ici les décisions de conception qui ont guidé le développement de la plateforme. Nous détaillons successivement l'architecture globale orientée microservices, l'architecture en couches du backend Spring Boot, la structure du frontend React.js et enfin l'organisation du moteur de recommandation IA basé sur Python FastAPI.

2.1 Architecture globale

2.1.1 Vue d'ensemble

L'architecture de LearnAgent suit une approche **orientée microservices** avec une séparation claire entre le frontend, le backend principal et le service de recommandation IA. Le système est composé de trois composantes indépendantes communicant via HTTP/REST, comme le montre la figure 2.1 :

1. **elearning-frontend** : Application SPA (Single Page Application) développée avec React.js 19 + Vite, s'exécutant sur le port **5173**.
2. **elearning-backend** : API RESTful développée avec Spring Boot 3.2.5 (Java 21) sur le port **8081**, exposant l'ensemble des endpoints métier.
3. **ai-recommendation-engine** : Microservice Python FastAPI dédié à l'IA sur le port **8000**, responsable des recommandations et de la détection des lacunes.

2.1.2 Diagramme d'architecture

La figure 2.1 ci-dessous illustre l'architecture globale du système, en mettant en évidence les flux de communication entre les trois composantes principales et la base de données PostgreSQL hébergée sur Supabase.

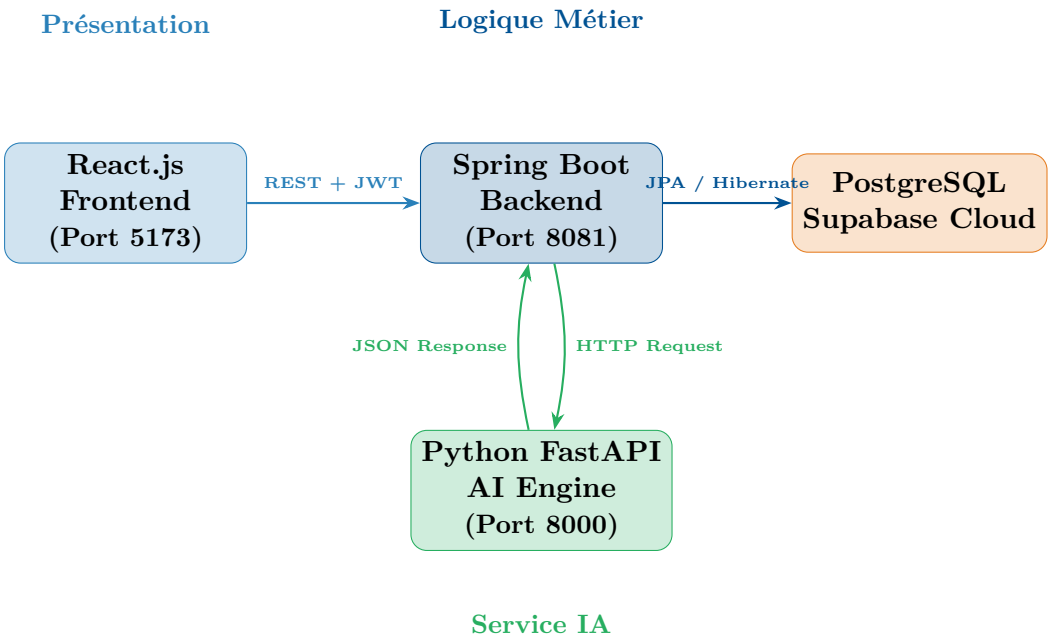

FIGURE 2.1 – Architecture globale du système LearnAgent

2.1.3 Diagramme de cas d'utilisation – Authentification

La figure 2.2 ci-dessous présente le diagramme de cas d'utilisation relatif au module d'authentification, détaillant les interactions entre les utilisateurs et le système pour les opérations de connexion, d'inscription et de récupération de compte.

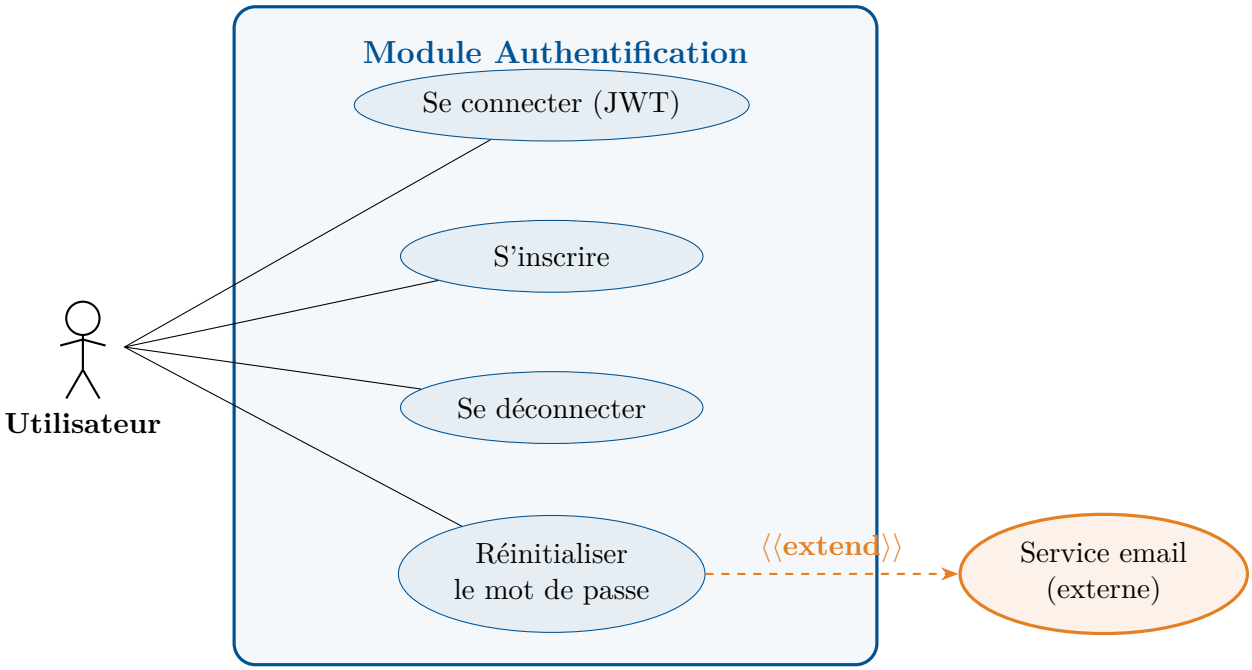

FIGURE 2.2 – Diagramme de cas d'utilisation – Module Authentification

2.2 Architecture du Backend (Spring Boot)

2.2.1 Structure en couches N-Tier

Le backend suit une architecture N-Tier avec séparation stricte des responsabilités, comme le montre la figure 2.3. Chaque couche a un rôle bien défini et ne communique qu'avec les couches adjacentes :

- **controller/** : Exposition des endpoints REST, validation des entrées, délégation aux services.
- **service/** : Logique métier, orchestration, appels WebClient vers le service IA.
- **repository/** : Interfaces Spring Data JPA, requêtes SQL générées par Hibernate.
- **model/** : 17 entités JPA mappées sur PostgreSQL.
- **dto/** : Objets de transfert (Request/Response) pour découpler le modèle de la présentation.
- **security/** : Filtre JWT (`JwtAuthenticationFilter`), gestion des rôles, `SecurityConfig`.
- **config/** : CORS, WebClient builder, serveur de fichiers statiques.
- **exception/** : Gestion globale des erreurs (`GlobalExceptionHandler` avec `@ControllerAdvice`).

Une composante importante de la couche service est l'**EmailService**, qui s'appuie sur le starter Spring Mail pour gérer les modèles HTML (`Thymeleaf`) et distribuer les notifications asynchrones.

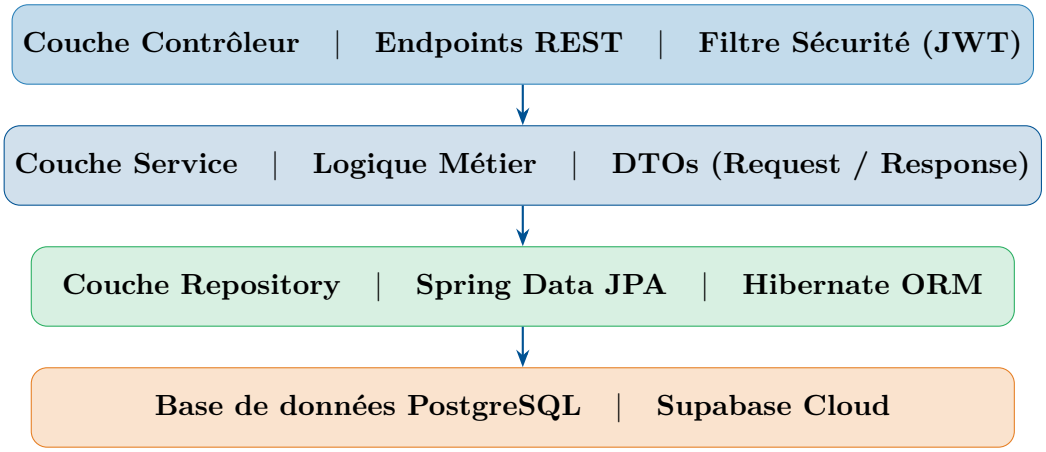

FIGURE 2.3 – Architecture N-Tier du backend Spring Boot

2.2.2 Modèle de données (17 entités JPA)

Le modèle de données de la plateforme est constitué de 17 entités JPA, regroupées en cinq domaines fonctionnels majeurs pour une meilleure clarté :

— **Utilisateurs & Sécurité :** `User` (profil et statut actif) et `Role` (niveaux d'accès : ADMIN, TEACHER, STUDENT).
— **Catalogue de Cours :** `Course` (informations principales et mots-clés sémantiques), `Chapter` (leçons ordonnées), `Category` (classification globale) et `SupportType` (formats supportés : PDF, vidéo...).
— **Suivi de l'Apprentissage :** `Enrollment` (gestion des inscriptions) et `ChapterProgress` (suivi strict de l'avancement servant de condition d'accès).
— **Évaluations :** `Quiz`, `QuizQuestion` et `QuizResult` pour les QCM conditionnels, ainsi que `Exercise` et `ExerciseCompletion` pour les devoirs pratiques.
— **Moteur IA & Interactions :** `Recommendation` (historique des affinités IA), `SearchHistory` (traces de recherche), `CourseRating` (avis) et `Message` (log des notifications emails).

2.2.3 Sécurité – Architecture JWT

La sécurité de la plateforme est assurée par un système d'authentification robuste basé sur les **JSON Web Tokens (JWT)**. Cette approche sans état (stateless) permet de vérifier l'identité des utilisateurs à chaque requête HTTP de manière performante, sans nécessiter de stockage de session côté serveur.

Concrètement, lors de la connexion, un token crypté et limité dans le temps est délivré au client. Ce token est ensuite transmis automatiquement dans l'en-tête de chaque requête vers le backend. Le serveur intercepte ce token, vérifie son intégrité, et autorise l'accès aux différentes fonctionnalités selon les privilèges associés au rôle de l'utilisateur (Étudiant, Enseignant ou Administrateur).

2.3 Architecture du Frontend (React.js)

2.3.1 Structure de l'application

Le frontend est développé avec React.js 19 et Vite, suivant une architecture modulaire orientée composants. La figure 2.4 donne un aperçu de l'organisation des répertoires principaux :

— **/api** : Couche d'abstraction Axios (`axiosConfig.js` avec intercepteurs JWT automatiques).
— **/components/common** : `Navbar.jsx` (navigation adaptative par rôle), `ProtectedRoute.jsx` (guard RBAC).
— **/components/course** : `CourseCard.jsx` (composant de carte de cours réutilisable).

— **/context** : `AuthContext.jsx` – état global d'authentification (JWT + localStorage).

— **/pages/auth** : `LoginPage.jsx`, `RegisterPage.jsx`.

— **/pages/student** : `Dashboard.jsx`, `CoursesPage.jsx`, `CourseViewPage.jsx` (chapitres progressifs), `QuizPage.jsx`, `ExercisesPage.jsx`, `SearchPage.jsx`.

— **/pages/teacher** : `TeacherDashboard.jsx`, `CreateCourse.jsx`, `CreateQuiz.jsx`, `CreateExercise.jsx`.

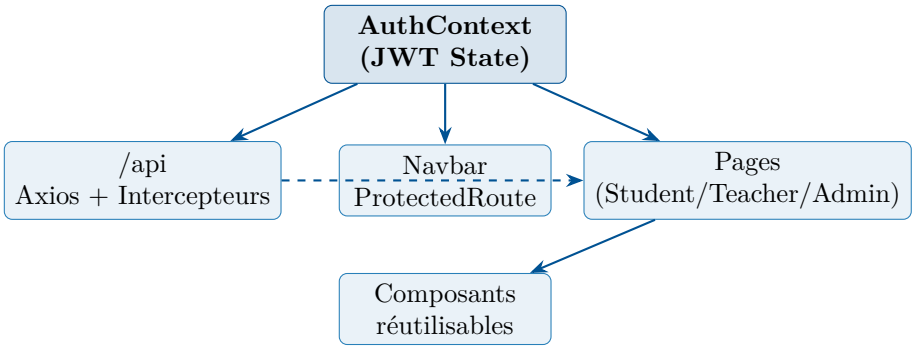

FIGURE 2.4 – Architecture modulaire du frontend React.js

2.3.2 Design System

L'interface utilise un **Dark Mode** avec les tokens de design suivants :

— **Palette** : Indigo/Violet (primaire) + Teal (accent), dégradés radiaux animés.

— **Glassmorphism** : `backdrop-filter: blur(20px)` sur les cartes et modals.

— **Typographie** : Police Inter (Google Fonts).

— **Animations** : `fadeInUp` staggeré, hover `translateY(-4px)`.

— **Responsive** : Breakpoint 768px, grilles CSS adaptatives.

2.4 Architecture du Moteur IA (Python FastAPI)

2.4.1 Structure du service

Le moteur de recommandation est un microservice Python indépendant (`ai-recommendation-engine/`), dont l'architecture est présentée à la figure 2.5. Il est composé des modules suivants :

— **main.py** : Point d'entrée FastAPI avec configuration CORS et enregistrement des routes.

— **models/recommender.py** : Classe `CourseRecommender` – chargement Sentence-BERT, pipeline d'embedding et de recommandation.

— **models/nlp_processor.py** : `NLPProcessor` – extraction de mots-clés, détection de langue (FR/EN).

— **services/text_extractor.py** : Extraction de texte PDF (PyPDF2), DOCX (python-docx), PPTX (python-pptx).

— **services/weak_topic_detector.py** : Détection des lacunes par similarité sémantique entre questions ratées et chapitres.

— **routes/recommendation_routes.py** : 6 endpoints REST exposés au backend Spring Boot.

— **schemas/recommendation_schema.py** : Modèles Pydantic pour la validation des requêtes/réponses.

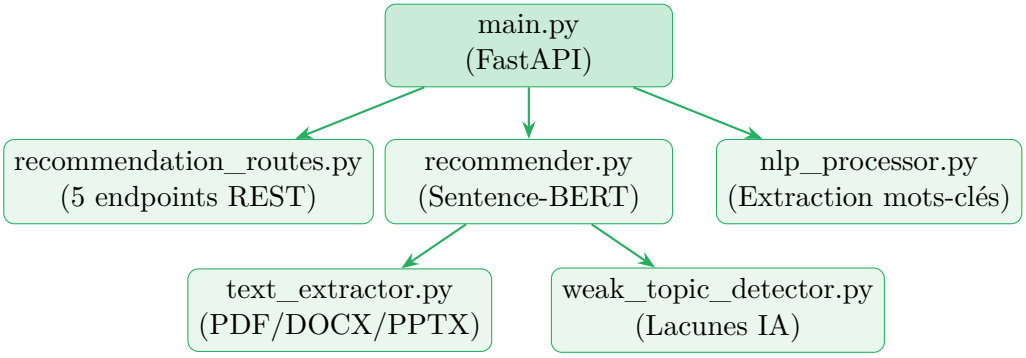

FIGURE 2.5 – Architecture du microservice Python FastAPI (moteur IA)

Conclusion

Ce deuxième chapitre a présenté l'architecture complète du système **LearnAgent**, articulée autour de trois composantes principales : un frontend React.js, un backend Spring Boot et un microservice IA Python FastAPI. Les choix architecturaux, notamment l'approche microservices, la séparation en couches N-Tier et l'utilisation de JWT pour la sécurité, ont été motivés par des impératifs de performance, de scalabilité et de maintenabilité. Les diagrammes présentés tout au long de ce chapitre offrent une vision claire et structurée du système, posant ainsi les bases pour les chapitres suivants qui détailleront la réalisation sprint par sprint.

Chapitre 3

Mise en œuvre incrémentale de la plateforme LearnAgent

Introduction

Ce chapitre retrace l'ensemble des cycles de développement de la plateforme **LearnAgent**, structurés selon la méthodologie agile Scrum en itérations courtes à valeur fonctionnelle croissante. Le premier sprint a posé les fondations sécuritaires de la plateforme, le second a construit le noyau pédagogique incluant la gestion des cours et le suivi de progression, tandis que le troisième s'est concentré sur les mécanismes d'évaluation formelle des acquis. En introduisant des modules interactifs de quiz et de soumission d'exercices, le système s'est progressivement enrichi d'outils d'évaluation automatique indispensables pour mesurer les performances de l'apprenant et offrir aux enseignants un suivi analytique complet de leurs classes.

3.1 Sprint 1 – Authentification et gestion des utilisateurs

Le premier sprint, réalisé sur deux semaines, a permis d'ériger le socle de sécurité de la plateforme. Les travaux ont porté sur l'authentification par jeton JWT, la gestion des utilisateurs structurée par rôles (Administrateur, Enseignant, Étudiant) et l'implémentation des interfaces de connexion et d'inscription. Le backend a été assuré principalement par **FRIKHA Amin**, tandis que le frontend a été pris en charge par **BAHLOUL Donia**.

3.1.1 Planification du sprint

Le backlog du sprint 1 a été organisé en trois volets complémentaires : la sécurité backend, la gestion métier des utilisateurs et l'interface d'authentification. La partie backend a couvert la configuration avancée de Spring Security, la génération et la validation des jetons JWT, ainsi que l'exposition des points d'accès d'authentification (`/api/auth/login`, `/api/auth/register`). En parallèle, la modélisation des entités utilisateurs, l'inscription

sécurisée avec hachage des mots de passe, et les fonctions d'administration ont été développées. Côté client, le frontend a intégré les pages d'authentification, le contexte React pour la persistance de session, la protection des routes par rôle, et la configuration d'Axios avec des intercepteurs globaux.

Table 3.1 – Sprint Backlog – Sprint 1 (Authentification & Sécurité)

ID	User Story	Priorité	Complexité
US 1.1	**En tant que** visiteur, **je souhaite** pouvoir créer un compte avec une adresse email valide **afin d'** accéder à la plateforme.	Haute	5 pts
US 1.2	**En tant qu'** utilisateur, **je souhaite** m'authentifier de manière sécurisée **afin d'** obtenir un jeton d'accès (JWT).	Haute	8 pts
US 1.3	**En tant qu'** administrateur, **je souhaite** disposer d'un espace de gestion **afin d'** attribuer et révoquer les rôles.	Moyenne	5 pts
US 1.4	**En tant que** système, **je dois** sécuriser les routes (API et Frontend) selon les privilèges associés au rôle **afin de** garantir la confidentialité.	Haute	8 pts

3.1.2 Réalisations techniques

L'architecture de sécurité mise en œuvre repose sur une chaîne d'approbation complète liant l'interface React, le contrôleur d'authentification, les services métier et la couche de persistance. Lors de la validation des identifiants, un jeton JWT est généré et transmis de manière sécurisée au client. Côté frontend, la session est centralisée au sein d'un contexte global (`AuthContext`), facilitant la gestion du profil utilisateur et l'application systématique d'un contrôle d'accès basé sur les rôles (RBAC).

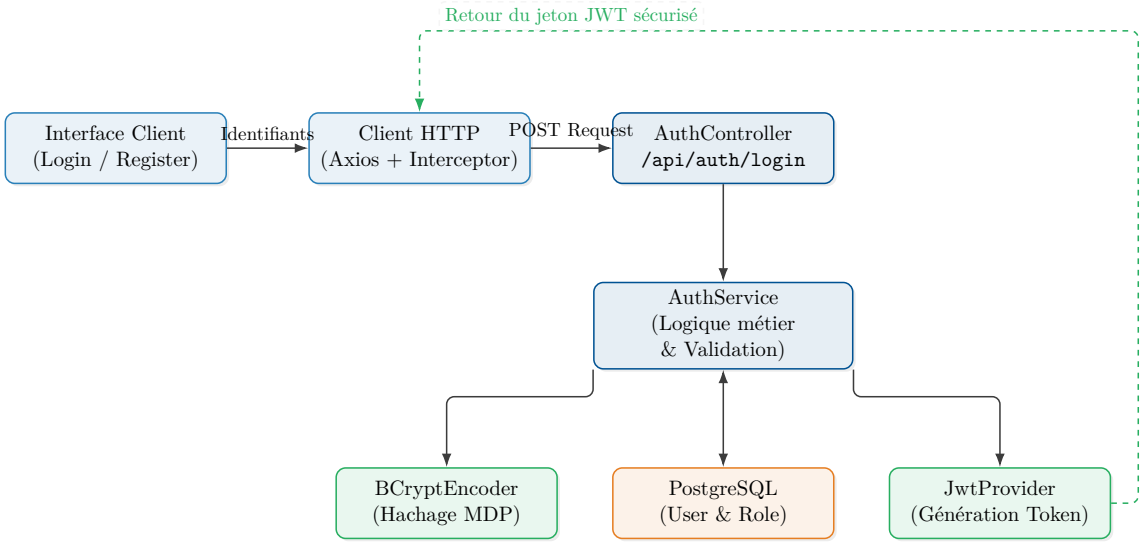

FIGURE 3.1 – Architecture détaillée du mécanisme d'authentification

Le contrôle des sessions s'effectue dynamiquement côté client. Les requêtes interceptées qui retournent une erreur 401 (Non autorisé) déclenchent automatiquement une invalidation de la session et une redirection vers la page de connexion.

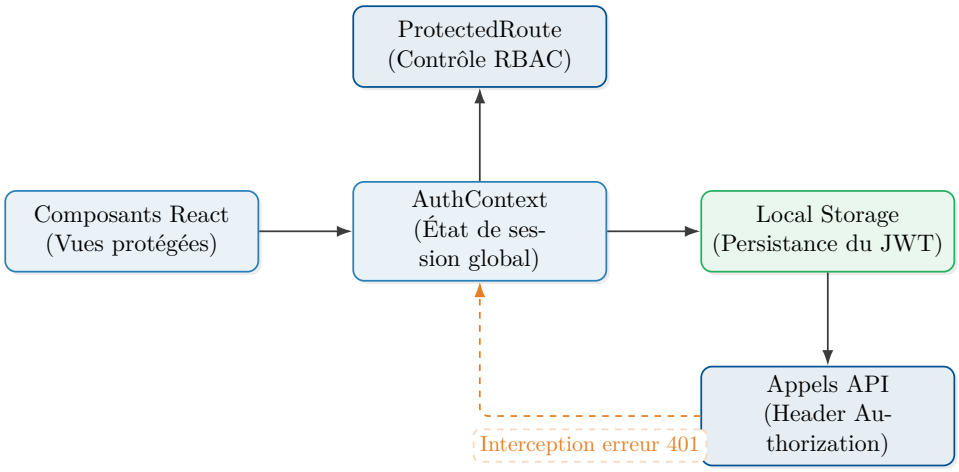

FIGURE 3.2 – Gestion des sessions et contrôle d'accès (Frontend)

3.1.3 Bilan du sprint

Les résultats de ce premier sprint garantissent la fiabilité et la sécurité de l'accès à LearnAgent. Le mécanisme d'inscription intégrant le chiffrement robuste des mots de passe (BCrypt) et l'authentification sans état (stateless) via JWT assurent une scalabilité optimale. Cette base permet une gestion granulaire des droits (`ADMIN`, `TEACHER`, `STUDENT`) garantissant la protection des ressources critiques aussi bien au niveau de l'API Spring Boot que des routes React.

Captures d'écran – Sprint 1

FIGURE 3.3 – Capture d'écran : Page d'inscription (Register)

FIGURE 3.4 – Capture d'écran : Page de connexion (Login)

FIGURE 3.5 – Capture d'écran : Tableau de bord Administrateur – Gestion des rôles

3.2 Sprint 2 – Gestion des cours multi-chapitres

Le deuxième sprint s'est focalisé sur le cœur pédagogique de LearnAgent. Il a permis d'implémenter la gestion hiérarchique des cours, le suivi rigoureux de la progression des apprenants et le contrôle d'accès conditionnel aux différents modules d'apprentissage. La logique métier backend a été conçue par **ABDELHEDI Wassim**, tandis que l'intégration et l'interface utilisateur ont été assurées par **BAHLOUL Donia**.

3.2.1 Planification du sprint

La modélisation du sprint 2 s'est articulée autour de trois axes majeurs : la conception de l'arborescence des cours et chapitres, le stockage des supports pédagogiques, et le moteur de calcul de progression. Les tâches ont couvert la création des entités relationnelles, le développement des API de gestion de fichiers (upload de vidéos MP4, PDF, etc.), le calcul dynamique de l'avancement, et les interfaces permettant aux enseignants de construire leurs cours.

Table 3.2 – Sprint Backlog – Sprint 2 (Moteur de Cours)

ID	User Story	Priorité	Complexité
US 2.1	**En tant qu'** enseignant, **je souhaite** créer et structurer un cours en plusieurs chapitres ordonnés **afin d'** offrir un apprentissage progressif.	Haute	8 pts
US 2.2	**En tant qu'** enseignant, **je souhaite** enrichir mes chapitres en y attachant divers supports (vidéos, PDF) **afin d'** améliorer l'engagement.	Haute	5 pts
US 2.3	**En tant qu'** étudiant, **je souhaite** m'inscrire à un cours et visualiser mon avancement en temps réel **afin de** suivre mon apprentissage.	Haute	5 pts
US 2.4	**En tant qu'** étudiant, **je dois** valider le chapitre en cours avant de débloquer le suivant **afin de** respecter le cheminement imposé.	Moyenne	3 pts

3.2.2 Réalisations techniques

Le modèle de données repose sur une relation hiérarchique stricte : un cours (`Course`) agrège une séquence ordonnée de chapitres (`Chapter`). Chaque chapitre héberge un type de média (`SupportType`) influençant son rendu frontend. Le mécanisme de progression (`ChapterProgress`) assure la liaison entre l'inscription de l'étudiant (`Enrollment`) et les chapitres, déclenchant des calculs automatiques du pourcentage global d'avancement.

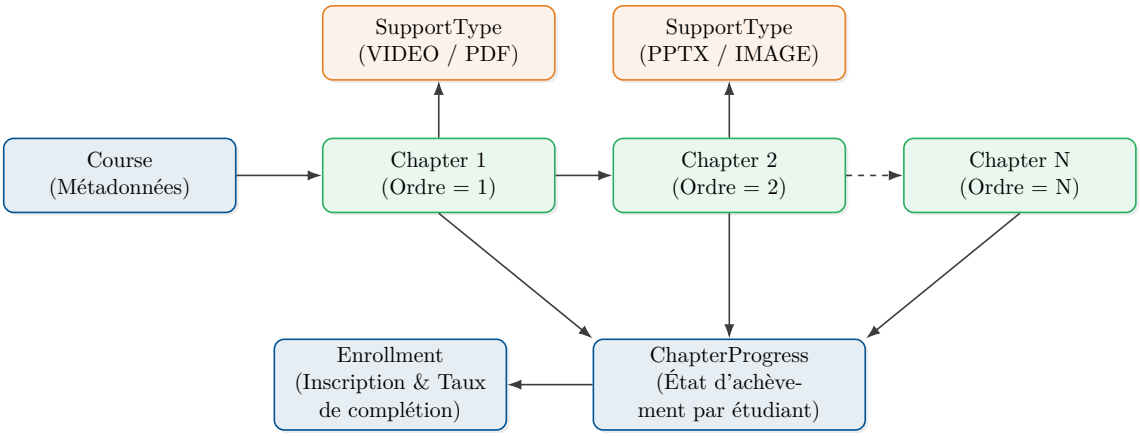

FIGURE 3.6 – Architecture relationnelle des cours multi-chapitres et suivi de progression

Le moteur de progression conditionne l'accès aux activités pratiques (Quiz, Exercices). Le système de vérification (`EnrollmentService`) garantit qu'un étudiant ne peut accéder à l'évaluation finale qu'après avoir marqué la totalité des chapitres comme terminés, garantissant ainsi l'intégrité du parcours pédagogique.

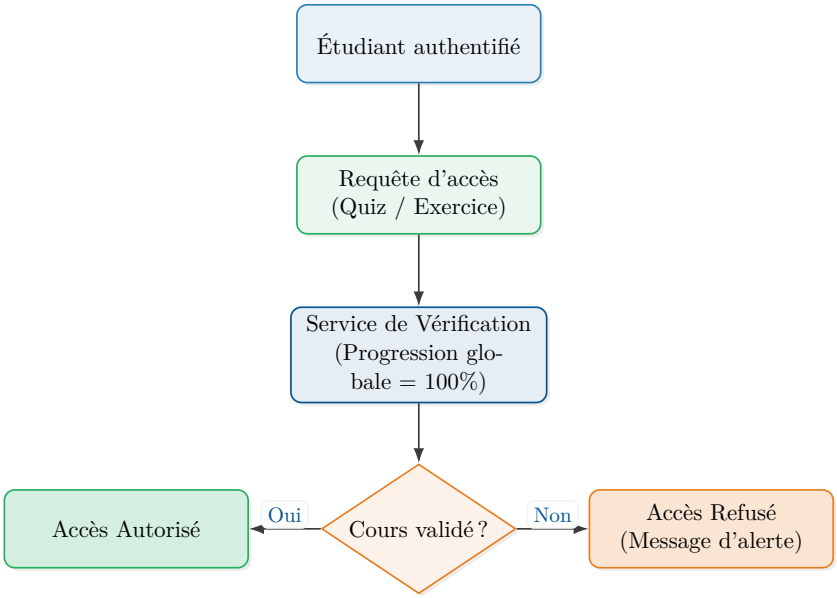

FIGURE 3.7 – Algorithme de contrôle d'accès basé sur la progression de l'étudiant

3.2.3 Bilan du sprint

La finalisation du sprint 2 dote LearnAgent d'un noyau pédagogique puissant et résilient. L'architecture supporte dorénavant la gestion de cours modulaires et la diffusion de ressources multimédias lourdes de façon fluide. Le tracking des chapitres s'est avéré extrêmement précis, générant des jauges de progression fiables sur les tableaux de bord étudiants. De plus, la règle d'affaire imposant l'achèvement à 100% avant toute évaluation

a été strictement implémentée et testée avec succès, consolidant la crédibilité académique de la plateforme.

Captures d'écran – Sprint 2

FIGURE 3.8 – Capture d'écran : Interface de création d'un cours (vue Enseignant)

FIGURE 3.9 – Capture d'écran : Visualisation des chapitres et jauge de progression (vue Étudiant)

FIGURE 3.10 – Capture d'écran : Message de blocage d'accès (progression incomplète)

3.3 Sprint 3 – Fonctionnalités pédagogiques avancées

Faisant suite au développement de l'architecture de base, ce sprint est centré sur les mécanismes d'évaluation formelle des acquis. En introduisant des modules interactifs de quiz et de soumission d'exercices, le système s'enrichit d'outils d'évaluation automatique indispensables pour mesurer les performances de l'apprenant et offrir aux enseignants un suivi analytique complet de leurs classes.

3.3.1 Planification du sprint

Le backlog du sprint 3 a ciblé la conception des évaluations asynchrones. Il englobe la création des structures de données complexes pour les quiz, les logiques de correction automatisée, et la gestion des livrables pratiques. La charge de développement a été répartie stratégiquement : **FRIKHA Amin** a piloté l'ingénierie des quiz et du scoring, **ABDELHEDI Wassim** s'est chargé du module d'exercices pratiques, et **BAHLOUL Donia** a assuré la création d'interfaces ergonomiques et réactives pour une expérience d'évaluation fluide.

Table 3.3 – Sprint Backlog – Sprint 3 (Quiz & Évaluation)

ID	User Story	Priorité	Complexité
US 3.1	**En tant qu'** enseignant, **je souhaite** concevoir des quiz (QCM) **afin d'** évaluer la compréhension des apprenants.	Haute	8 pts
US 3.2	**En tant qu'** étudiant, **je souhaite** passer un quiz interactif en fin de parcours **afin de** valider mes acquis.	Haute	5 pts
US 3.3	**En tant qu'** enseignant, **je souhaite** publier des exercices pratiques avec pièces jointes **afin de** diversifier l'évaluation.	Moyenne	5 pts
US 3.4	**En tant qu'** étudiant, **je souhaite** soumettre un exercice et marquer mon travail comme complété **afin de** mettre à jour mon dossier.	Moyenne	3 pts
US 3.5	**En tant qu'** enseignant, **je souhaite** accéder à un tableau de bord analytique **afin de** suivre la réussite de mes étudiants.	Basse	5 pts

3.3.2 Module d'Évaluation : Quiz

Architecture du module

L'implémentation du module de quiz repose sur un graphe d'entités interconnectées. L'entité centrale `Quiz` qualifie l'épreuve (niveau de difficulté, métadonnées). Elle est liée à une collection d'items `QuizQuestion` définissant l'énoncé, les distracteurs (options) et la clé de correction. L'entité `QuizResult` historise l'ensemble des tentatives de l'apprenant, constituant la source de données primaire pour les statistiques et le futur système de recommandation par IA.

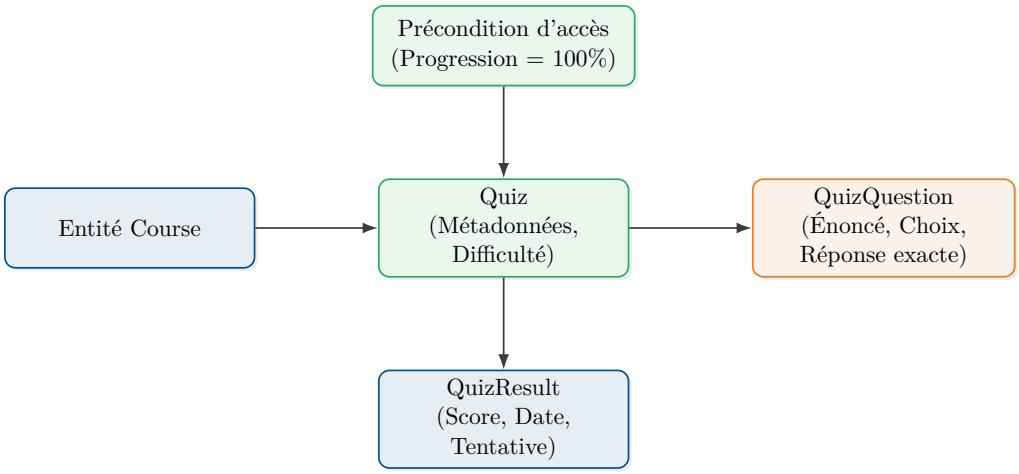

FIGURE 3.11 – Modèle architectural du module d'évaluation par Quiz

Soumission et correction algorithmique

Le flux de traitement d'un quiz garantit la fiabilité et l'instantanéité des retours. À la soumission, un pipeline transactionnel valide d'abord l'éligibilité de l'étudiant, confronte les réponses soumises aux clés de correction, agrège les points, calcule le pourcentage de réussite, et persiste l'entité `QuizResult` en base de données.

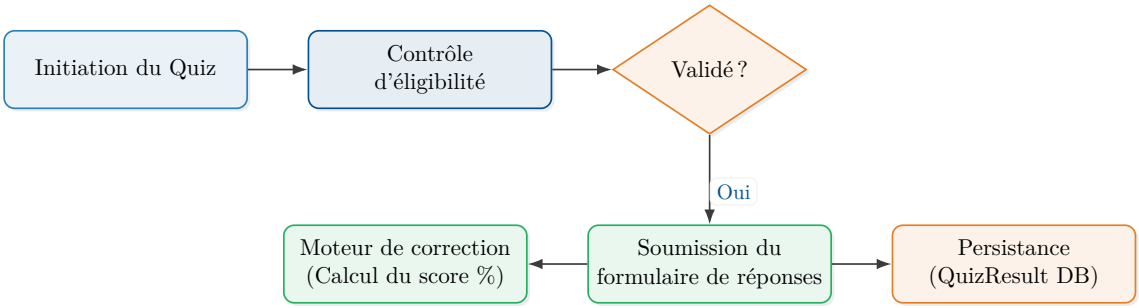

FIGURE 3.12 – Pipeline transactionnel de correction automatique d'un Quiz

3.3.3 Module d'Activités Pratiques : Exercices

Architecture du module

En complément de l'évaluation théorique, le module des exercices adresse les compétences pratiques. L'entité `Exercise` encapsule un sujet détaillé, un niveau de complexité et un support didactique téléchargeable. Le traçage de la réalisation s'effectue via l'entité de jonction `ExerciseCompletion`, alimentant directement les tableaux de bord de supervision des enseignants.

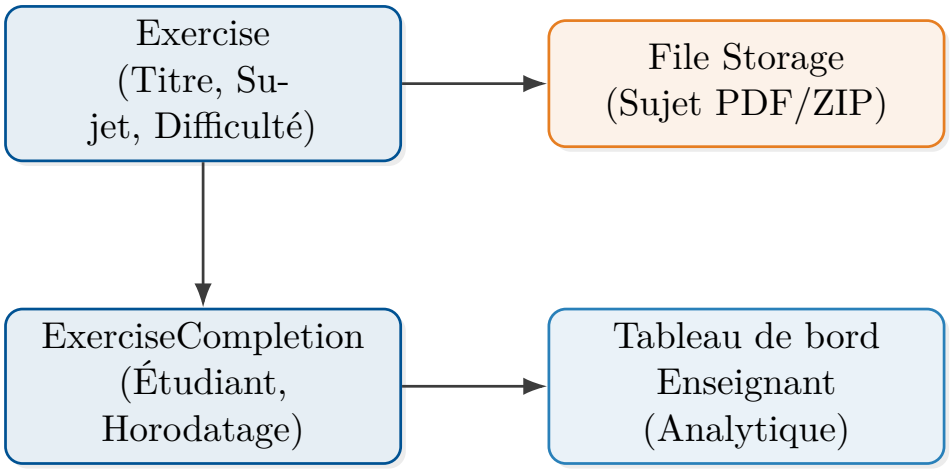

FIGURE 3.13 – Logique de liaison et de suivi du module Exercice

3.3.4 Bilan du sprint

L'achèvement du sprint 3 marque un tournant qualitatif pour la plateforme LearnAgent. La mise en production du moteur d'évaluation QCM s'est accompagnée du développement d'un algorithme de correction instantané à la fois performant et robuste face aux cas de figure anormaux. La restriction d'accès aux évaluations a été couplée au moteur de progression (Sprint 2) pour consolider l'expérience d'apprentissage séquentiel. Par ailleurs, la gestion fluide des supports d'exercices et des états de complétion permet aujourd'hui de restituer aux enseignants un profil analytique riche et exploitable, valorisant ainsi considérablement l'écosystème pédagogique.

Captures d'écran – Sprint 3

FIGURE 3.14 – Capture d'écran : Interface de création d'un quiz QCM (vue Enseignant)

FIGURE 3.15 – Capture d'écran : Passage d'un quiz interactif (vue Étudiant)

FIGURE 3.16 – Capture d'écran : Résultat et score après soumission du quiz

FIGURE 3.17 – Capture d'écran : Interface de soumission d'un exercice pratique (vue Étudiant)

FIGURE 3.18 – Capture d'écran : Tableau de bord analytique Enseignant (suivi des étudiants)

Conclusion

Ces trois sprints ont permis de construire incrémentalement une plateforme pédagogique complète et cohérente. L'intégration de la sécurité JWT couplée à un moteur de structuration de cours performant forme une base logicielle mature. La boucle d'apprentissage est désormais complète : structuration de cours, diffusion de contenus, suivi de progression, et évaluation des connaissances. Sur cette base logicielle exhaustive et éprouvée, le chapitre suivant s'attachera à l'intégration d'une couche d'intelligence artificielle (moteur Sentence-BERT), visant à propulser LearnAgent vers un modèle d'apprentissage adaptatif de nouvelle génération.

Chapitre 4

Sprints 4 et 5 – Intelligence Artificielle et Messagerie Avancée

Introduction

Ce chapitre couvre les deux derniers sprints du projet. Le Sprint 4 intègre un moteur de recommandation sémantique basé sur Sentence-BERT, une détection automatique des lacunes et un tuteur pédagogique Gemini, déployés sous forme d'un microservice Python FastAPI indépendant. Le Sprint 5 met en place un système de messagerie assurant la récupération sécurisée des comptes et l'envoi de notifications automatiques aux apprenants.

4.1 Sprint 4 – Intelligence Artificielle et Recommandation Sémantique

Ce quatrième sprint, d'une durée de deux semaines, a été mené conjointement par ABDELHEDI Wassim pour le développement du moteur IA et FRIKHA Amin pour son intégration au backend. Son objectif principal consistait à concevoir un moteur de recommandation hybride exploitant le modèle Sentence-BERT, enrichi de mécanismes automatiques de détection des lacunes et d'un tuteur pédagogique interactif.

4.1.1 Architecture du Service IA et Choix technologique : Sentence-BERT

Le service de recommandation repose sur le modèle `paraphrase-multilingual-MiniLM-L12-v2` issu de la bibliothèque *Sentence-Transformers*. Il s'agit d'une architecture Transformer BERT fine-tunée pour la similarité sémantique, capable de produire des vecteurs de 384 dimensions couvrant plus de 50 langues. Son avantage majeur réside dans la recherche cross-lingue : une requête soumise en français peut retrouver des cours rédigés en anglais et vice-versa. Contrairement à une recherche classique par mots-clés, qui ne retourne que des correspondances exactes, Sentence-BERT évalue la proximité sémantique globale des

énoncés, rendant la découverte de ressources beaucoup plus naturelle et pertinente pour l'apprenant.

4.1.2 Sprint Backlog – Sprint 4

Les onze tâches planifiées pour ce sprint ont été réparties selon trois domaines de responsabilité. ABDELHEDI Wassim a pris en charge l'ensemble du développement du microservice IA : configuration de l'environnement FastAPI, module d'extraction textuelle multi-format, processeur NLP bilingue, moteur de recommandation hybride, détecteur de lacunes sémantique et service tuteur Gemini. FRIKHA Amin a assuré la couche d'intégration Spring Boot, comprenant les endpoints REST, le service de recherche WebClient asynchrone et la persistance de l'historique. BAHLOUL Donia a développé l'interface React avec la page de recherche multi-entités et l'affichage des recommandations dans le tableau de bord étudiant.

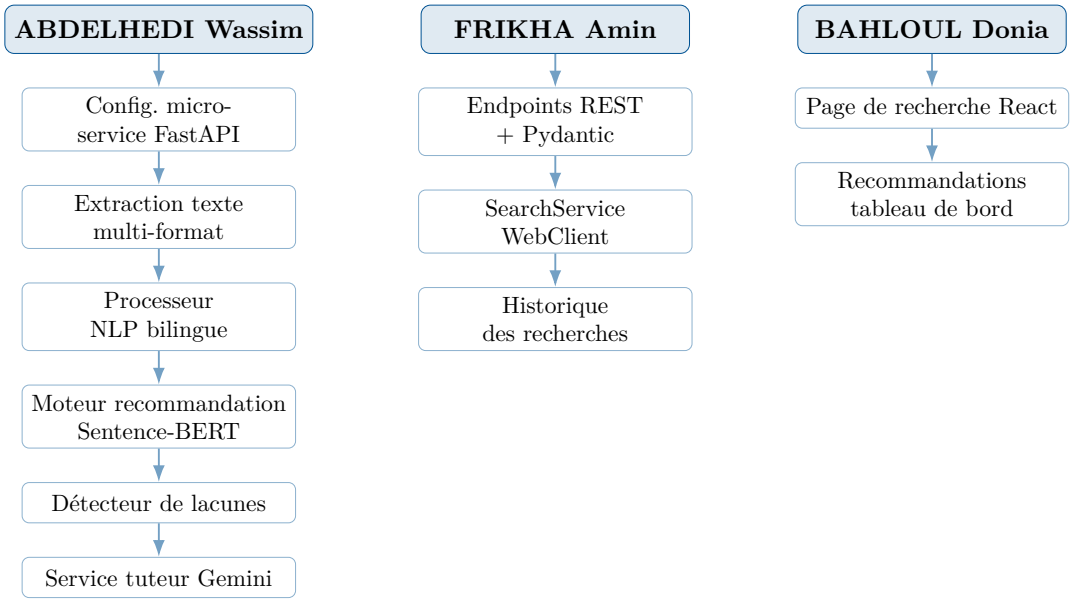

FIGURE 4.1 – Répartition des tâches – Sprint Backlog Sprint 4

4.2 Pipeline de Recommandation Hybride

4.2.1 Flux complet de recommandation

Le pipeline de recommandation combine une analyse sémantique profonde à des ajustements contextuels pour affiner les résultats. Comme illustré dans le schéma ci-dessous, la requête de l'utilisateur traverse six étapes successives, de l'extraction des mots-clés jusqu'à la sélection finale des cours les plus pertinents.

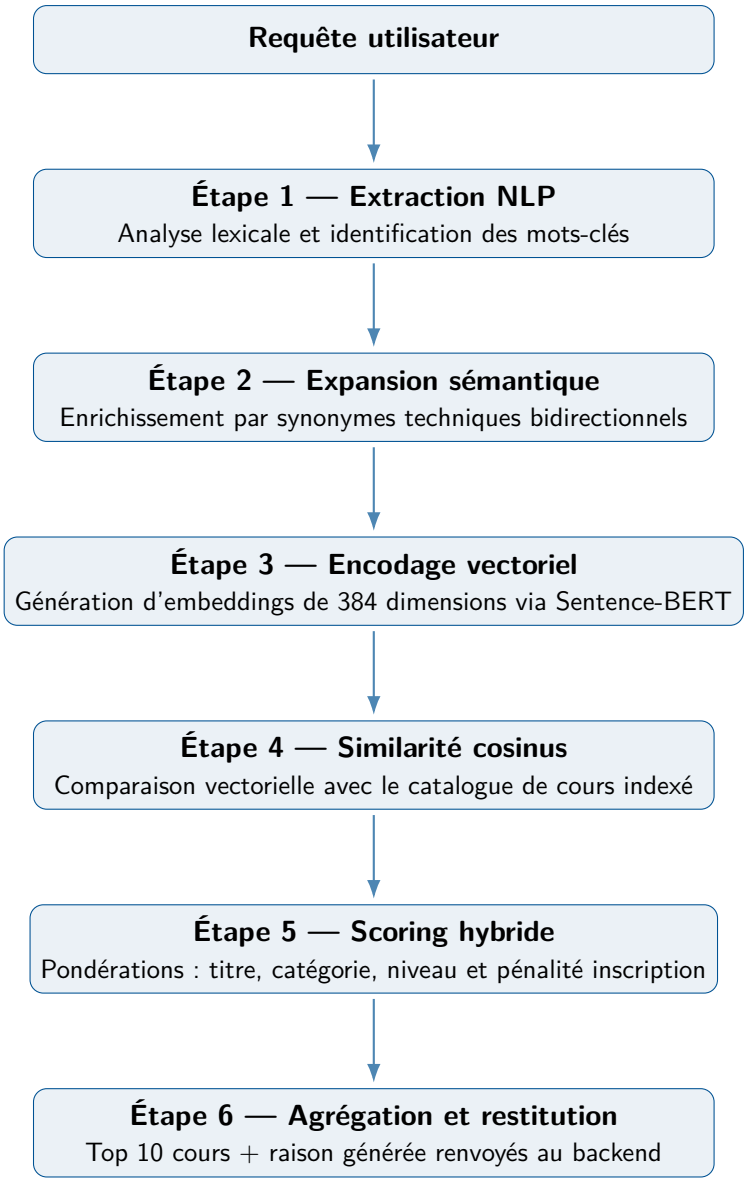

FIGURE 4.2 – Schéma explicatif du flux complet de recommandation hybride LearnAgent

4.2.2 Algorithme de Scoring Hybride

Le score global attribué à chaque cours résulte de la combinaison de sa similarité cosinus avec la requête enrichie et de plusieurs pondérations contextuelles. Des bonifications sont accordées lorsque la requête correspond au titre, à la catégorie ou au niveau de difficulté ciblé par l'apprenant. À l'inverse, une pénalité multiplicative est appliquée si l'étudiant est déjà inscrit à ce cours afin de favoriser la découverte de nouveaux contenus. Seuls les cours dépassant un seuil minimal de pertinence sont conservés, et les scores sont plafonnés à une valeur maximale unitaire.

4.2.3 Expansion de Requête et NLP Bilingue

Le module d'analyse linguistique enrichit automatiquement les requêtes courtes grâce à un dictionnaire de synonymes techniques bidirectionnels. Ainsi, un sigle ou un terme générique est étendu à ses variantes et acronymes associés pour maximiser les correspondances pertinentes. Ce traitement inclut également une détection heuristique de la langue de saisie, un filtrage des mots vides bilingue et une extraction de l'intention de l'utilisateur, permettant au moteur d'ajuster dynamiquement le score de pertinence selon le niveau de difficulté recherché.

4.2.4 Intégration Backend (SearchService)

Le service de recherche du backend Spring Boot orchestre l'intégralité du flux transactionnel. Il agrège le catalogue des cours publiés et les données d'inscription de l'utilisateur, puis initie un appel asynchrone non-bloquant vers le microservice IA. Un mécanisme de secours automatique (fallback) bascule vers une recherche textuelle classique en base de données en cas d'inaccessibilité du moteur IA, garantissant ainsi la continuité du service. L'ensemble des requêtes est tracé dans un historique persistant, et les recommandations produites alimentent de manière proactive l'interface personnalisée de l'étudiant.

4.3 Détection des Lacunes (Weak Topic Detector)

4.3.1 Algorithme de détection

L'algorithme de détection des lacunes effectue une analyse sémantique ciblée des erreurs commises lors d'une évaluation. Après un échec à un quiz, le système encode le contenu textuel de chaque chapitre du cours ainsi que l'énoncé de chaque question incorrecte. Une comparaison par similarité cosinus associe chaque erreur au chapitre traitant le sujet correspondant, sous réserve que le score de proximité dépasse un seuil minimal. Le taux d'erreur est ensuite calculé par chapitre et classé selon trois niveaux de sévérité — critique, moyen et faible — aboutissant à la génération d'un rapport d'orientation indiquant à l'étudiant les sections précises à réviser en priorité.

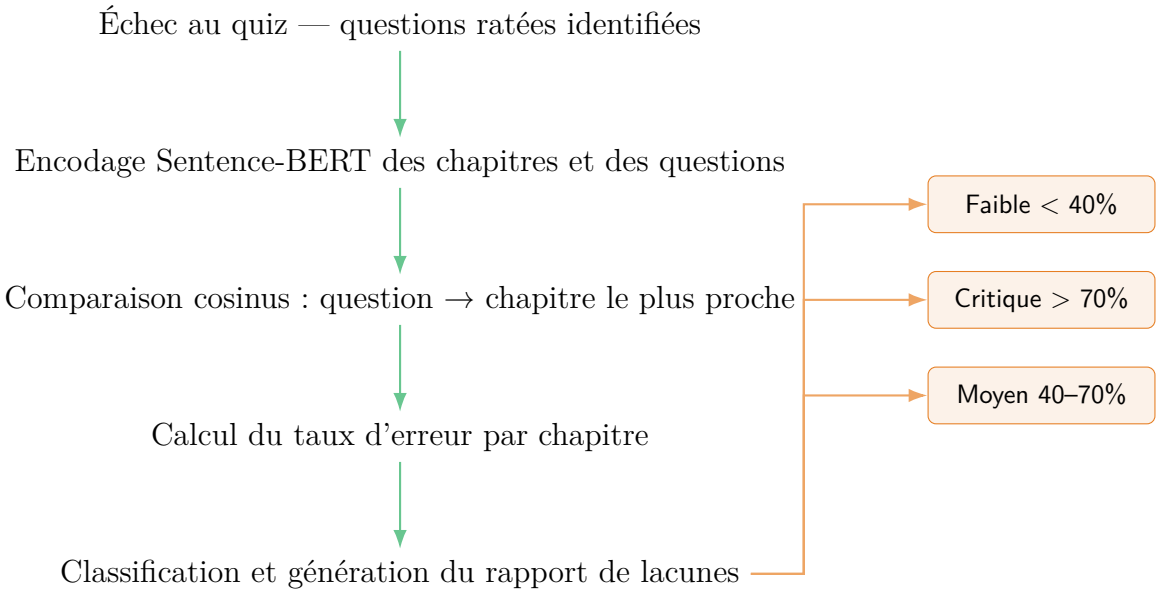

FIGURE 4.3 – Schéma du processus de détection automatique des lacunes pédagogiques

4.4 Tuteur Pédagogique IA (Gemini)

Afin de proposer un retour formatif personnalisé, un tuteur intelligent basé sur l'API Google Gemini complète le dispositif de détection des lacunes. Lorsqu'une erreur est identifiée, le service transmet la question, la réponse de l'étudiant et la correction attendue au modèle de langage. Ce dernier génère une explication concise et constructive, guidant l'apprenant vers la compréhension du concept sans se limiter à lui livrer la réponse exacte. Pour optimiser les performances, l'ensemble des questions incorrectes d'un quiz est traité en une seule requête groupée (batch), réduisant drastiquement le temps d'attente global.

4.5 Endpoints REST du Service IA

Le microservice FastAPI expose une interface RESTful structurée autour des fonctionnalités clés de la couche IA. Le schéma suivant présente les principaux points d'accès et leur rôle respectif au sein de la plateforme.

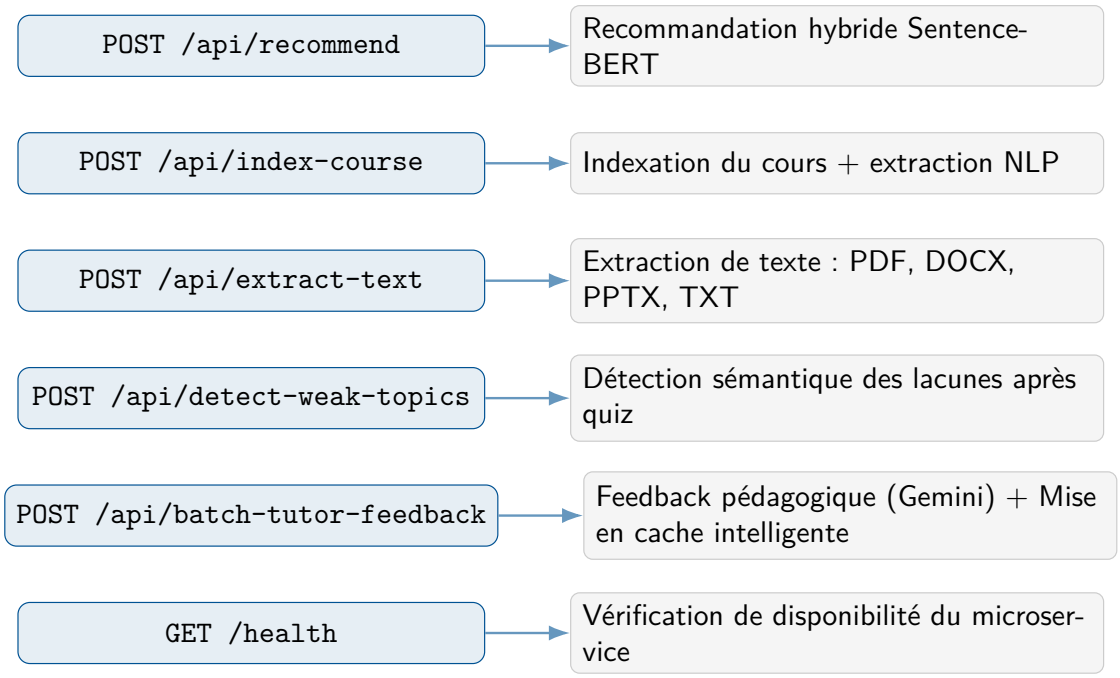

FIGURE 4.4 – Endpoints REST du microservice IA FastAPI

4.5.1 Tableau de Bord Analytique et Visualisation de Performance

Afin d'offrir une vision à 360 degrés de la progression de l'étudiant, le Sprint 4 a été enrichi d'un module d'analytique avancée. Ce dispositif transforme les données brutes de quiz en indicateurs de performance (KPIs) exploitables, visualisés à travers un tableau de bord haute fidélité accessible aux **enseignants** et aux **administrateurs**.

Sprint

Indicateurs de performance (KPIs) calculés côté backend :

- **Moyenne Globale** : Calculée en temps réel sur l'ensemble des tentatives de quiz.
- **Taux de Réussite** : Ratio entre quiz validés et tentatives totales.
- **Évolution de Tendance** : Algorithme comparant le score du dernier quiz à la moyenne glissante des sessions précédentes pour dégager un indice de progression (+/-).

Visualisation de données avec Recharts

L'interface intègre la bibliothèque `Recharts` pour le rendu d'une **courbe de performance adaptative** (Area Chart). Ce graphique est doté d'une intelligence visuelle : la couleur de la courbe s'ajuste dynamiquement (vert pour la progression, rouge pour la régression). Une fonctionnalité majeure est la **comparaison sémantique avec la classe**,

où une ligne de référence superposée permet à l'étudiant de se situer instantanément par rapport à la moyenne globale de ses pairs pour chaque évaluation.

Moteur d'Insights Pédagogiques (IA)

En complément des graphiques, un processeur logique côté backend analyse les métriques pour générer des **insights textuels automatiques**. Ces phrases courtes (ex : "Forte progression constatée", "Performance régulière") offrent un feedback immédiat à l'apprenant, simulant la présence d'un tuteur humain capable de synthétiser son parcours d'apprentissage.

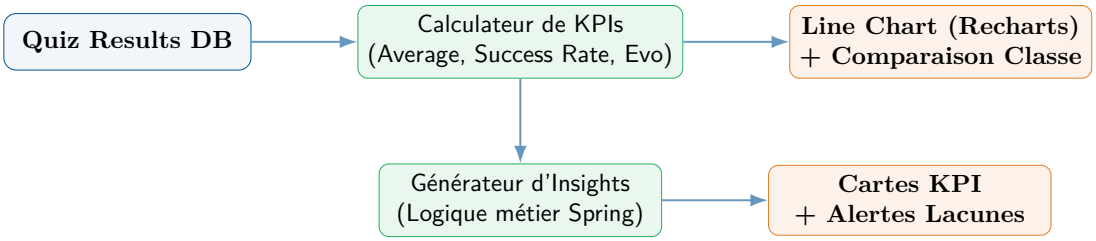

FIGURE 4.5 – Architecture du moteur analytique et de visualisation des performances

4.6 Sprint Review – Sprint 4

La clôture du Sprint 4 valide l'intégration réussie d'une couche d'intelligence artificielle autonome au sein de la plateforme. Le moteur de recommandation hybride a démontré sa capacité à relier sémantiquement des requêtes multilingues à un catalogue de formations diversifié. Une innovation majeure a été apportée au tuteur pédagogique via l'implémentation d'un système de **mise en cache intelligente** (Smart Caching) des explications en base de données, réduisant la latence à zéro pour les questions déjà analysées. De plus, une **chaîne de secours multi-modèles** (Gemini 2.0 Flash, Pro, etc.) garantit une résilience totale face aux limitations de quota. L'isolation du microservice IA et ces optimisations garantissent des performances stables et une expérience utilisateur fluide, validant ainsi la pertinence d'une approche microservices. Par ailleurs, l'extension des outils analytiques aux administrateurs assure une supervision globale et cohérente de la réussite académique au sein de l'établissement.

4.7 Sprint 5 – Système de Messagerie Intelligente

Ce cinquième et dernier sprint, d'une durée d'une semaine, a été mené par FRIKHA Amin pour le développement backend et BAHLOUL Donia pour la conception des interfaces. Son objectif visait à mettre en place un écosystème de communication avancé, intégrant la validation des comptes, la récupération sécurisée des mots de passe et l'envoi de notifications automatiques liées à l'activité pédagogique.

4.7.1 Architecture du module de messagerie

Le dispositif repose sur un service d'envoi d'emails adossé au protocole SMTP, générant des courriels dynamiques à partir de templates HTML personnalisés. Il orchestre la création de jetons uniques et temporaires pour sécuriser les procédures de réinitialisation de mot de passe. Des tâches planifiées en arrière-plan scrutent l'activité de la plateforme afin de relancer automatiquement les étudiants inactifs. En outre, le système réagit aux événements applicatifs : toute modification apportée par un enseignant à un cours déclenche l'envoi d'une alerte informative à l'ensemble des inscrits concernés.

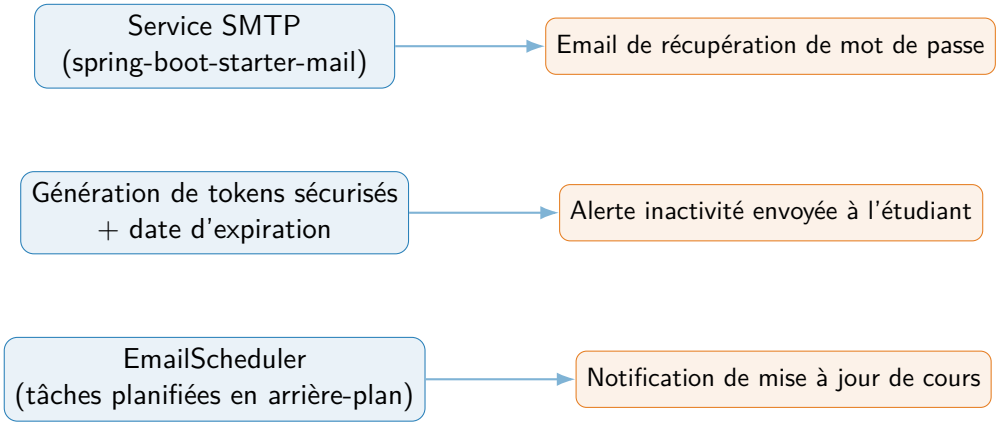

FIGURE 4.6 – Architecture du module de messagerie – Sprint 5

4.7.2 Intégration Frontend

Côté client, l'expérience utilisateur a été enrichie de nouvelles interfaces ergonomiques dédiées à la gestion de l'identité numérique. Les flux de réinitialisation de mot de passe s'intègrent naturellement au design system existant, permettant à l'utilisateur de récupérer ses accès de manière autonome, intuitive et totalement sécurisée, sans compromettre l'architecture d'authentification JWT existante.

Conclusion

Ce chapitre a détaillé la réalisation des deux ultimes sprints du projet, scellant l'aboutissement technique de la plateforme LearnAgent. L'intégration du moteur de recommandation sémantique Sentence-BERT et du tuteur pédagogique Gemini dote la plateforme d'une réelle capacité d'adaptation aux besoins de chaque apprenant. Associée au système de notification intelligent du Sprint 5, la solution offre désormais une couverture fonctionnelle globale : de la diffusion du savoir à l'évaluation personnalisée, en passant par le suivi individualisé et l'engagement continu des étudiants.

Références Bibliographiques

[1] K. Schwaber, J. Sutherland, *The Scrum Guide – The Definitive Guide to Scrum*, Novembre 2020. Disponible sur : `https://scrumguides.org/`.

[2] Spring Framework Documentation, *Spring Boot Reference Guide – Version 3.2.x*, Disponible sur : `https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/`, Consulté en 2025.

[3] Spring Framework Contributors, *Spring Security Reference Documentation*, VMware, Inc., 2023. Disponible sur : `https://docs.spring.io/spring-security/reference/`.

[4] Meta Open Source, *React 19 – A JavaScript library for building user interfaces*, Disponible sur : `https://react.dev/`, Consulté en 2025.

[5] M. Jones, J. Bradley, N. Sakimura, *JSON Web Token (JWT) – RFC 7519*, Internet Engineering Task Force (IETF), Mai 2015.

[6] N. Reimers, I. Gurevych, *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*, Proceedings of EMNLP 2019. Disponible sur : `https://arxiv.org/abs/1908.10084`.

[7] HuggingFace, *Model : paraphrase-multilingual-MiniLM-L12-v2*, Disponible sur : `https://huggingface.co/sentence-transformers/`, Consulté en 2025.

[8] G. Salton, C. Buckley, *Term-weighting approaches in automatic text retrieval*, Information Processing & Management, Vol. 24, No. 5, pp. 513–523, 1988.

[9] S. Ramírez, *FastAPI – Modern, fast web framework for building APIs with Python*, Disponible sur : `https://fastapi.tiangolo.com/`, Consulté en 2025.

[10] F. Pedregosa et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research, Vol. 12, pp. 2825–2830, 2011.

[11] PostgreSQL Global Development Group, *PostgreSQL 15 Documentation*, Disponible sur : `https://www.postgresql.org/docs/15/`, Consulté en 2025.

[12] E. You, *Vite – Next Generation Frontend Tooling*, Disponible sur : `https://vitejs.dev/`, Consulté en 2025.

[13] Supabase Inc., *Supabase – The Open Source Firebase Alternative*, Disponible sur : `https://supabase.com/`, Consulté en 2025.

[14] M. Fowler, *UML Distilled : A Brief Guide to the Standard Object Modeling Language*, 3e éd., Addison-Wesley Professional, 2003.

Chapitre A

Tableau récapitulatif des Endpoints REST Backend

Table A.1: Principaux endpoints REST du backend Spring Boot (port 8081)

Méthode **Rôle**	**Endpoint**	**Description**
POST Public	/api/auth/register	Inscription
POST Public	/api/auth/login	Connexion – retourne
GET Authentifié	/api/auth/me	Infos utilisateur conne
GET Public	/api/courses	Liste des cours publiés
POST TEACHER	/api/courses	Créer un cours
PUT TEACHER	/api/courses/{id}	Modifier un cours
DELETE TEACHER	/api/courses/{id}	Supprimer un cours
GET TEACHER	/api/courses/my-courses	Mes cours (enseignant
GET Authentifié	/api/courses/{id}/chapters	Chapitres d'un cours
POST STUDENT	/api/enrollments/{courseId}	S'inscrire à un cours
POST	/api/chapters/{id}/complete	Marquer chapitre com

Table A.1 – suite de la page précédente

Méthode **Rôle**	**Endpoint**	**Description**
STUDENT		
GET Public	/api/quizzes	Liste des quiz publiés
POST TEACHER	/api/quizzes	Créer un quiz
POST STUDENT	/api/quizzes/{id}/submit	Soumettre un quiz
POST STUDENT	/api/search	Recherche IA (cours)
POST IA / Service	/api/detect-weak-topics	Détection lacunes
GET ADMIN	/api/admin/users	Liste des utilisateurs
GET ADMIN / TEACHER	/api/admin/users/{id}	Détails analytiques étu
GET ADMIN	/api/admin/stats	Statistiques globales

Chapitre B

Stack Technologique

Table B.1 – Technologies utilisées dans le projet LearnAgent

Couche	**Technologie**	**Rôle**
Backend	Spring Boot 3.2.5	API REST, sécurité, accès données
Backend	Spring Security + JWT	Authentification et gestion des rôles
Backend	JPA / Hibernate	ORM et persistance
Base de données	PostgreSQL 15	Stockage relationnel
Cloud DB	Supabase	Hébergement PostgreSQL managé
Frontend	React 19 + Vite	Interface utilisateur SPA
Frontend	Context API	Gestion d'état globale
Frontend	Dark Mode Design System	Charte graphique de la plateforme
Service IA	Python FastAPI	Microservice de recommandation
Service IA	Sentence-Transformers	Embeddings sémantiques multilingues
Service IA	Scikit-learn	Calcul de similarité cosinus
Service IA	Google Gemini API	Tuteur pédagogique + Smart Caching
Messagerie	Spring Mail (SMTP)	Envoi d'emails transactionnels
Gestion projet	Scrum / Jira	Méthodologie agile

LearnAgent

Plateforme E-Learning Intelligente

Étudiants :
Wassim ABDELHEDI
Donia BAHLOUL
Amin FRIKHA

Encadrante :
Mme. Sahla MASMOUDI

École Nationale d'Ingénieurs de Sfax
Année Universitaire 2025-2026

Conclusion Générale et Perspectives

Au terme de ce Projet de Fin d'Année, nous avons conçu et développé **LearnAgent**, une plateforme e-learning intelligente et complète répondant à l'ensemble des besoins identifiés lors de l'analyse de l'existant. Structuré en cinq sprints Scrum successifs, le projet a permis de livrer de façon incrémentielle un système d'authentification JWT sécurisé avec gestion des rôles, un moteur de cours multi-chapitres avec progression stricte, des modules interactifs de quiz et d'exercices pratiques, puis un moteur de recommandation sémantique Sentence-BERT couplé à un tuteur pédagogique Gemini, et enfin un système robuste de messagerie et de notifications automatiques.

Sur le plan des compétences, ce projet nous a permis de consolider notre maîtrise des architectures microservices et de la conception d'API RESTful, en combinant Spring Boot 3.2 pour le backend transactionnel, React 19 pour le frontend et Python FastAPI pour la couche d'intelligence artificielle. Nous avons approfondi la gestion de la sécurité applicative (JWT, BCrypt, RBAC), le traitement du langage naturel multilingue avec Sentence-BERT et la similarité cosinus, ainsi que l'intégration des grands modèles de langage via l'API Gemini. Au-delà des aspects techniques, ce projet nous a également permis de renforcer notre capacité à travailler en équipe, à gérer les priorités dans un cadre agile et à documenter rigoureusement nos travaux.

En perspective, plusieurs axes d'amélioration se dégagent naturellement de ce premier prototype fonctionnel. L'intégration de modèles de langage de grande taille (GPT-4, Gemini) permettrait de générer des contenus pédagogiques personnalisés et d'enrichir le tuteur virtuel. Un moteur d'apprentissage adaptatif dynamique, ajustant le parcours de chaque étudiant en temps réel selon ses performances, constituerait une avancée significative. Par ailleurs, le déploiement d'une application mobile React Native, l'ajout de fonctionnalités sociales (forums, sessions collaboratives) et la mise en place de mécanismes de gamification (badges, classements, points de progression) contribueraient à renforcer l'engagement des apprenants. Enfin, la containerisation Docker et l'orchestration Kubernetes sur un hébergement cloud garantiraient la scalabilité et la résilience de la plateforme à grande échelle.

BAHLOUL Donia, FRIKHA Amin, ABDELHEDI Wassim
Sfax, 2026